

Chapter 6 Polymorphism

CHAPTER OBJECTIVES

- Define polymorphism and explore its benefits.
- Discuss the concept of dynamic binding.
- Use inheritance to create polymorphic references.
- Explore the purpose and syntax of Java interfaces.
- Use interfaces to create polymorphic references.
- Discuss object-oriented design in the context of polymorphism.

This chapter examines polymorphism — another foundational principle of object-oriented software. We begin by exploring the concept of binding and its relationship to polymorphism. We then examine the two distinct mechanisms Java provides for creating polymorphic references: inheritance and interfaces. Java interfaces are explored in their own right, with attention to how they resemble abstract classes and how they complete the polymorphism picture. The chapter concludes with a discussion of the design considerations that arise when applying polymorphism.

6.1 Late Binding

Typically, the type of a reference variable exactly matches the class of the object it refers to. Consider the following declaration:

```
ChessPiece bishop;
```

The `bishop` variable would naturally point to an object created from the `ChessPiece` class. However, it does not have to. The type of a reference variable and the type of the object it points to must be compatible — but they need not be identical. The relationship is more flexible than an exact match.

The term polymorphism means "having many forms." A polymorphic reference is a reference variable that can point to different types of objects at different moments during program execution. The specific method that gets invoked through a polymorphic reference — the actual code that runs — can change from one call to the next.

Consider this line of code:

```
obj.doIt();
```

If `obj` is a polymorphic reference, it can point to different types of objects at different times. If this line appears inside a loop, or within a method that is called repeatedly, it may invoke a different version of `doIt` on each occasion.

The act of committing to a particular method definition in response to an invocation is called binding — specifically, binding a method invocation to a method definition. For many method calls, this commitment can be made at compile time. For polymorphic references, however, it cannot. The appropriate method definition can only be determined at run time, based on the actual type of the object being referenced at the moment of the call. This deferred commitment is referred to as late binding or dynamic binding. It carries a small runtime overhead compared to compile-time binding, but that cost is generally well worth the flexibility that polymorphic references provide.

In Java, polymorphic references can be created in two ways: through inheritance and through interfaces. We examine each approach in turn.

KEY CONCEPT

A polymorphic reference can refer to different types of objects over time.

KEY CONCEPT

The binding of a method invocation to its definition is performed at run time for a polymorphic reference.

6.2 Polymorphism via Inheritance

When a reference variable is declared using a particular class name, it can hold a reference to any object of that class. Beyond that, it can also refer to any object whose class is related to the declared type through inheritance. If `Mammal` is the parent of `Horse`, for example, a `Mammal` reference can legally point to a `Horse` object:

```
Mammal pet;  
Horse secretariat = new Horse();  
pet = secretariat; // a valid assignment
```

This might initially appear to conflict with the principle of strong typing introduced in Chapter 2, but it does not. Strong typing requires that a variable only be assigned a value consistent with its declared type — and that is precisely what is happening here. Inheritance establishes an is-a relationship: a horse is-a mammal. Assigning a `Horse` object to a `Mammal` reference is therefore entirely consistent with the declared type.

The reverse assignment — from a `Mammal` reference to a `Horse` reference — is also possible but requires an explicit cast. This direction is generally less useful and more prone to problems, because while a horse has all the capabilities of a mammal, the reverse does not necessarily hold.

KEY CONCEPT

A reference variable can refer to any object created from any class related to it by inheritance.

This flexibility extends throughout a class hierarchy. If `Mammal` is itself derived from a class called `Animal`, the following is also valid:

```
Animal creature = new Horse();
```

Taken to its logical conclusion, an `Object` reference can point to any object at all, since every class in Java ultimately descends from `Object`.

The reference variable `creature` can be used polymorphically because at any point in time it may refer to an `Animal`, a `Mammal`, or a `Horse` object. Suppose all three classes define a method called `move`, each overriding the previous definition with its own implementation.

The following invocation:

```
creature.move();
```

calls the `move` method, but which version executes is determined at run time based on what `creature` actually refers to at that moment.

KEY CONCEPT

The type of the object, not the type of the reference, determines which version of a method is invoked.

If `creature` refers to an `Animal` object, the `Animal` version of `move` executes. If it refers to a `Mammal`, the `Mammal` version runs. If it refers to a `Horse`, the `Horse` version is called.

Abstract classes present no obstacle to polymorphism. Suppose `Mammal` is abstract, and `move` is defined as an abstract method in `Mammal` that is concretely implemented in `Horse`, `Dog`, and `Whale` (all derived from `Mammal`). A `Mammal` reference can still be used to refer to objects of any of these concrete classes and to invoke `move` polymorphically on each of them, despite the fact that `Mammal` itself cannot be instantiated.

The `Firm` class in Listing 6.1 provides a `main` driver that creates a `Staff` of employees and invokes the `payday` method to process their compensation. The output includes identifying information about each employee and how much they are paid.

The `Staff` class in Listing 6.2 maintains an array of objects representing the individual employees. The array is declared to hold `StaffMember` references, but it is populated with objects created from several different subclasses — `Executive`, `Employee`, `Hourly`, and `Volunteer`. Because all of these classes descend from `StaffMember`, the assignments are entirely valid. The `staffList` array is an array of polymorphic references.

The `payday` method of `Staff` iterates through the employee list, prints each employee's information, and invokes the `pay` method on each one to determine their compensation. The call to `pay` is polymorphic — each subclass provides its own implementation of the method.

The `StaffMember` class in Listing 6.3 is declared abstract. It does not represent any particular type of employee and is never instantiated directly. Instead, it acts as the common ancestor of all employee subclasses, housing the state and behavior that all employees share: a name, address, and phone number, stored as `protected` instance variables.

`StaffMember` also defines a `toString` method that assembles these common fields into a readable string, and declares an abstract method called `pay` that takes no parameters and returns a `double`. Providing a concrete definition of `pay` at the `StaffMember` level would be meaningless — how an employee is paid depends entirely on their employment type — so the definition is deliberately left abstract for each subclass to fill in.

This example captures the essence of polymorphism. Each class in the hierarchy knows best how to handle payment for its specific type of employee, yet from the perspective of the `payday` method, all employees are simply being paid. Polymorphism allows us to treat related but distinct objects in a consistent manner while still permitting each to behave in its own appropriate way.

By declaring `pay` as abstract in `StaffMember`, the `payday` method of `Staff` can invoke it through a `StaffMember` reference without ambiguity. If `pay` had not been declared in `StaffMember` at all, the compiler would reject the invocation, because it would have no way to verify that every object stored in `staffList` is guaranteed to have a `pay` method.

The `Volunteer` class in Listing 6.4 represents a person who contributes their time without monetary compensation. It stores only the basic personal information, which is forwarded to the `StaffMember` constructor via `super`. The `pay` method simply returns `0.0`. Had `pay` not been overridden, `Volunteer` would have remained abstract and could not have been instantiated.

In the `payday` method of `Staff`, when a volunteer receives a zero pay amount, a simple message of thanks is printed. For all other employees whose pay value exceeds zero, the payment amount is displayed.

The `Employee` class in Listing 6.5 represents a salaried employee compensated at a fixed rate per pay period. In addition to the basic personal information, the constructor accepts a social security number and a pay rate, forwarding the personal data to `StaffMember` via `super`. The `toString` method is overridden to append the additional employment information to the string produced by the parent's `toString`, which is retrieved using `super.toString()`. The `pay` method simply returns the stored pay rate.

The `Executive` class in Listing 6.6 represents an employee who may receive a bonus on top of their base pay. Derived from `Employee`, it inherits from both `Employee` and `StaffMember`. The constructor delegates its arguments to the `Employee` constructor and initializes the bonus to zero.

Bonuses are awarded through the `awardBonus` method, which is called in the `payday` method of `Staff` for the single executive in the `staffList`. Since `awardBonus` exists only

in `Executive` and not in `StaffMember`, the `staffList` element must be explicitly cast to `Executive` before the method can be invoked.

The `pay` method of `Executive` first determines the base salary by calling `super.pay()` to invoke the `Employee` version, then adds the bonus. Using `super.pay()` rather than directly accessing `payRate` is the better design choice: if the way `Employee` objects are paid ever changes, `Executive` will automatically reflect that change without requiring any modification. After the combined payment is computed and returned, the bonus is reset to zero in preparation for the next pay period.

The `Hourly` class in Listing 6.7 represents an employee paid on an hourly basis. It tracks the number of hours worked in the current pay period through the `addHours` method, which is called from `payday`. The `pay` method computes payment as the product of the pay rate and hours worked, then resets the hours counter to zero. The `toString` method extends the parent's version to include the current hours worked.

```
// LISTING 6.1

//*****

// Firm.java

//

// Demonstrates polymorphism via inheritance.

//*****

public class Firm
{
    //-----

    // Creates a staff of employees for a firm and pays them.

    //-----

    public static void main (String[] args)
    {
        Staff personnel = new Staff();

        personnel.payday();
    }
}
```

```
}
```

OUTPUT

Name: Tony

Address: 123 Main Line

Phone: 555-0469

Social Security Number: 123-45-6789

Paid: 2923.07

Name: Paulie

Address: 456 Off Line

Phone: 555-0101

Social Security Number: 987-65-4321

Paid: 1246.15

Name: Vito

Address: 789 Off Rocker

Phone: 555-0000

Social Security Number: 010-20-3040

Paid: 1169.23

Name: Michael

Address: 678 Fifth Ave.

Phone: 555-0690

Social Security Number: 958-47-3625

Current hours: 40

Paid: 422.0

Name: Adrianna

Address: 987 Babe Blvd.

Phone: 555-8374

Thanks!

Name: Benny

Address: 321 Dud Lane

Phone: 555-7282

Thanks!

// LISTING 6.2

//*****

// Staff.java

//

// Represents the personnel staff of a particular business.

//*****

public class Staff

{

 private StaffMember[] staffList;

 //-----

 // Constructor: Sets up the list of staff members.

 //-----

```

public Staff ()
{
    staffList = new StaffMember[6];

    staffList[0] = new Executive ("Tony", "123 Main Line",
                                   "555-0469", "123-45-6789", 2423.07);
    staffList[1] = new Employee ("Paulie", "456 Off Line",
                                   "555-0101", "987-65-4321", 1246.15);
    staffList[2] = new Employee ("Vito", "789 Off Rocker",
                                   "555-0000", "010-20-3040", 1169.23);
    staffList[3] = new Hourly ("Michael", "678 Fifth Ave.",
                                   "555-0690", "958-47-3625", 10.55);
    staffList[4] = new Volunteer ("Adrianna", "987 Babe Blvd.",
                                   "555-8374");
    staffList[5] = new Volunteer ("Benny", "321 Dud Lane",
                                   "555-7282");

    ((Executive)staffList[0]).awardBonus (500.00);
    ((Hourly)staffList[3]).addHours (40);
}

//-----
// Pays all staff members.
//-----

public void payday ()
{
    double amount;

```



```

    for (int count=0; count < staffList.length; count++)
    {
        System.out.println (staffList[count]);

        amount = staffList[count].pay();    // polymorphic

        if (amount == 0.0)

            System.out.println ("Thanks!");

        else

            System.out.println ("Paid: " + amount);

        System.out.println ("-----");
    }
}
}

```

```

// LISTING 6.3

//*****

// StaffMember.java

//

// Represents a generic staff member.

//*****

abstract public class StaffMember
{

```

```
protected String name;

protected String address;

protected String phone;

//-----

// Constructor: Sets up this staff member using the specified
// information.

//-----

public Staff Member (String eName, String eAddress, String ePhone)
{
    name = eName;

    address = eAddress;

    phone = ePhone;
}

//-----

// Returns a string including the basic employee information.

//-----

public String toString()
{
    String result = "Name: " + name + "\n";

    result += "Address: " + address + "\n";

    result += "Phone: " + phone;

    return result;
}

//-----
```

```

// Derived classes must define the pay method for each type of
// employee.
//-----
public abstract double pay();
}

```

```

// LISTING 6.4
//*****
// Volunteer.java
//
// Represents a staff member that works as a volunteer.
//*****

public class Volunteer extends StaffMember
{
    //-----

    // Constructor: Sets up this volunteer using the specified
    // information.
    //-----

    public Volunteer (String eName, String eAddress, String ePhone)
    {
        super (eName, eAddress, ePhone);
    }

    //-----

    // Returns a zero pay value for this volunteer.

```

```
//-----  
  
public double pay()  
  
{  
  
    return 0.0;  
  
}  
  
}
```

```
// LISTING 6.5  
  
//*****  
  
// Employee.java  
  
//  
  
// Represents a general paid employee.  
  
//*****  
  
public class Employee extends StaffMember  
{  
  
    protected String socialSecurityNumber;  
  
    protected double payRate;  
  
  
    //-----  
  
    // Constructor: Sets up this employee with the specified  
  
    // information.  
  
    //-----  
  
    public Employee (String eName, String eAddress, String ePhone,  
  
                     String socSecNumber, double rate)  
  
    {
```

```

        super (eName, eAddress, ePhone);

        socialSecurityNumber = socSecNumber;

        payRate = rate;
    }

    //-----

    // Returns information about an employee as a string.

    //-----

    public String toString()
    {
        String result = super.toString();

        result += "\nSocial Security Number: " + socialSecurityNumber;

        return result;
    }

    //-----

    // Returns the pay rate for this employee.

    //-----

    public double pay()
    {
        return payRate;
    }
}

```

//LISTING 6.6

//*****

```

// Executive.java

//

// Represents an executive staff member, who can earn a bonus.

//*****

public class Executive extends Employee
{

    private double bonus;

    //-----

    // Constructor: Sets up this executive with the specified
    // information.
    //-----

    public Executive (String eName, String eAddress, String ePhone,
                      String socSecNumber, double rate)

    {

        super (eName, eAddress, ePhone, socSecNumber, rate);

        bonus = 0;    // bonus has yet to be awarded

    }

    //-----

    // Awards the specified bonus to this executive.
    //-----

    public void awardBonus (double execBonus)

    {

        bonus = execBonus;

    }
}

```

```

//-----
// Computes and returns the pay for an executive, which is the
// regular employee payment plus a one-time bonus.
//-----

public double pay()
{
    double payment = super.pay() + bonus;

    bonus = 0;

    return payment;
}
}

```

```

// LISTING 6.7

//*****

// Hourly.java

//

// Represents an employee that gets paid by the hour.

//*****

public class Hourly extends Employee
{
    private int hoursWorked;

    //-----

    // Constructor: Sets up this hourly employee using the specified

```

```

// information.

//-----

public Hourly (String eName, String eAddress, String ePhone,
               String socSecNumber, double rate)
{
    super (eName, eAddress, ePhone, socSecNumber, rate);
    hoursWorked = 0;
}

//-----

// Adds the specified number of hours to this employee's
// accumulated hours.
//-----

public void addHours (int moreHours)
{
    hoursWorked += moreHours;
}

//-----

// Computes and returns the pay for this hourly employee.
//-----

public double pay()
{
    double payment = payRate * hoursWorked;
    hoursWorked = 0;
    return payment;
}

```



```

//-----
// Returns information about this hourly employee as a string.
//-----

public String toString()
{
    String result = super.toString();

    result += "\nCurrent hours: " + hoursWorked;

    return result;
}
}

```

Here is the reconstructed text:

6.3 Interfaces

In Chapter 5 we used the term interface to describe the set of public methods through which a client interacts with an object. That meaning remains consistent with our usage here, but we now give it a formal definition as a Java language construct. Interfaces provide a second mechanism for creating polymorphic references.

KEY CONCEPT

An interface is a collection of abstract methods and therefore cannot be instantiated.

A Java interface is a collection of constants and abstract methods. As discussed in Chapter 8, an abstract method is one that carries no implementation — no body of code is provided, and the method header, including its parameter list, is simply followed by a semicolon. An interface cannot be instantiated.

Listing 6.8 shows an interface called `Encryptable` containing two abstract methods: `encrypt` and `decrypt`. Abstract methods in an interface may optionally be preceded by the `abstract` keyword, though in practice this is usually omitted. Methods declared within an interface have `public` visibility by default.

```
// LISTING 6.8

//*****

// Encryptable.java

//

// Represents the interface for an object that can be encrypted
// and decrypted.

//*****

public interface Encryptable
{

    public void encrypt();

    public String decrypt();

}
```

A class fulfills an interface contract by providing concrete implementations for every abstract method the interface declares. The `Secret` class in Listing 6.9 implements the `Encryptable` interface. A class signals its intent to fulfill an interface by including the `implements` keyword followed by the interface name in its class header. If the class fails to provide a concrete implementation for even one of the interface's methods, the compiler will flag the omission as an error.

In `Secret`, both `encrypt` and `decrypt` are fully implemented, satisfying the contract defined by `Encryptable`. These method implementations must carry the same signatures as the abstract declarations in the interface. In `Secret`, the encryption scheme is a simple Caesar cipher, which shifts each character of the message by a fixed number of positions. An entirely different class could implement `Encryptable` using a completely different encryption algorithm — the interface specifies what must be done, not how.

Note that `Secret` also defines methods beyond those required by `Encryptable`, specifically `isEncrypted` and `toString`. An interface establishes a minimum set of behaviors a class must provide; it does not prevent the class from offering additional functionality. In fact, classes that implement interfaces almost always define additional methods as well.

Listing 6.10 shows the `SecretTest` program, which creates and exercises several `Secret` objects.

The relationship between an interface and the class that implements it can be represented in a UML class diagram. An interface is depicted similarly to a class node, but with the designation `<<interface>>` placed above the interface name. A dotted arrow with a triangular arrowhead connects the implementing class to the interface. Figure 9.2 illustrates the UML class diagram for the `SecretTest` program.

Multiple classes can implement the same interface, each providing its own concrete definitions for the required methods. A `Password` class, for example, could also implement `Encryptable`, using whatever encryption scheme is appropriate for passwords. The interface mandates which methods must exist — it says nothing about how those methods must work.

A class may also implement more than one interface simultaneously. In such cases, the class must provide implementations for every method declared in every interface it claims to implement. The `implements` clause lists multiple interface names separated by commas:

```
class ManyThings implements Interface1, Interface2, Interface3
{
    // implements all methods of all interfaces
}
```

In addition to abstract methods, an interface may also contain constants declared with the `final` modifier. Any class that implements such an interface gains access to those constants automatically.

Interface Hierarchies

KEY CONCEPT

Inheritance can be applied to interfaces so that one interface can be derived from another.

Inheritance applies to interfaces just as it applies to classes. One interface can be derived from another, forming an interface hierarchy that parallels a class hierarchy in structure. In UML, inheritance relationships between interfaces are shown with the same open-arrowhead connection used for class inheritance.

When one interface derives from another, it inherits all of the abstract methods and constants of the parent interface. Any class that implements the child interface must provide concrete implementations for every inherited method as well as any methods added by the child interface. Visibility is not a complicating factor in interface hierarchies — all interface members are public by definition.

It is important to understand that class hierarchies and interface hierarchies are entirely separate structures. An interface cannot be used to derive a class, and a class cannot be used to derive an interface. The only point of contact between the two is when a class declares that it implements a particular interface.

Before turning to how interfaces support polymorphism, let's examine two widely used interfaces from the Java standard class library: `Comparable` and `Iterator`.

The `Comparable` Interface

The Java standard class library defines a number of interfaces alongside its classes. One of the most commonly encountered is the `Comparable` interface, found in the `java.lang` package. It declares a single method — `compareTo` — that accepts an object as a parameter and returns an integer value.

The purpose of `Comparable` is to establish a uniform mechanism for ordering objects relative to one another. One object invokes the method and passes another as the argument:

```
if (obj1.compareTo(obj2) < 0)
    System.out.println ("obj1 is less than obj2");
```

By the contract of the interface, the returned integer should be negative if the calling object is considered less than the parameter object, zero if they are considered equal, and positive if the calling object is considered greater. Each implementing class decides for itself what "less than," "equal to," and "greater than" mean for objects of that type.

The `Iterator` Interface

Another widely used interface from the Java standard class library is `Iterator`. It is designed for classes that represent a collection of objects and need to provide a standard means of moving through that collection one element at a time.

In Chapter 3 we introduced the concept of an iterator informally, using loops to work through a collection's elements. Most iterator objects in Java — including instances of the `Scanner` class — are formally defined using the `Iterator` interface.

The two primary methods of the `Iterator` interface are `hasNext`, which returns a `boolean`, and `next`, which returns an object. Neither takes any parameters. The `hasNext` method returns `true` as long as elements remain to be processed; `next` retrieves the subsequent object. The class implementing the interface determines the order in which elements are returned by `next`.

It is worth emphasizing that `next` does not remove the returned object from the underlying collection — it merely returns a reference to it. The `Iterator` interface also declares a `remove` method, which takes no parameters and has a `void` return type. Calling `remove` eliminates the most recently returned object from the underlying collection.

We have already used iterators extensively — with the `Scanner` class in Chapter 3. The for-each loop simplifies this processing in many situations. Iterators remain a central tool as we develop collection classes in later chapters of this book.

6.4 Polymorphism via Interfaces

KEY CONCEPT

An interface name can be used to declare an object reference variable.

Having explored interfaces in their own right, we can now examine how they enable polymorphic references. Just as a class name can be used to declare the type of a reference variable, an interface name can also serve as the type of a reference variable. An interface reference can point to any object of any class that implements that interface.

Suppose we declare an interface called `Speaker`:

```
public interface Speaker

{
    public void speak();
    public void announce (String str);
}
```

The name `Speaker` can now be used to declare a reference variable:

```
Speaker current;
```

KEY CONCEPT

An interface reference can refer to any object of any class that implements that interface.

The variable `current` can hold a reference to any object whose class implements `Speaker`. If we define a class called `Philosopher` that implements `Speaker`, we can assign a `Philosopher` object to a `Speaker` reference:

```
current = new Philosopher();
```

This assignment is valid because a `Philosopher` is-a `Speaker`. In this respect, the relationship between a class and an interface it implements mirrors the is-a relationship created through inheritance — and it is this relationship that forms the basis for polymorphism via interfaces.

If we additionally create a class called `Dog` that also implements `Speaker`, a `Dog` object can be assigned to the same reference variable. The variable can point to a `Philosopher` at one moment and a `Dog` the next:

```
Speaker guest;
```

```
guest = new Philosopher();
guest.speak();
guest = new Dog();
guest.speak();
```

The first call to `speak` invokes the `Philosopher` implementation; the second invokes the `Dog` implementation. As with inheritance-based polymorphism, it is the type of the object the reference currently points to — not the declared type of the reference — that determines which method executes.

One important constraint: when using an interface reference, only the methods declared in the interface may be invoked through that reference, even if the underlying object supports additional methods. Suppose `Philosopher` also defines a `public` method called `pontificate`. The following would produce a compiler error:

```
Speaker special = new Philosopher();
special.pontificate(); // generates a compiler error
```

The compiler knows only that `special` is a `Speaker` and can therefore guarantee only the methods the `Speaker` interface declares. Since `special` could at any point refer to a `Dog` (which has no `pontificate` method), the compiler disallows the call. If we know with certainty that the object is a `Philosopher` in a specific context, an explicit cast resolves this:

```
((Philosopher)special).pontificate();
```

KEY CONCEPT

A parameter to a method can be polymorphic, giving the method flexible control of its arguments.

An interface name can also serve as the declared type of a method parameter. Any object of any class implementing that interface can then be passed as an argument to the method. For example:

```
public void sayIt (Speaker current)
{
    current.speak();
}
```

Both a `Dog` and a `Philosopher` can be passed to `sayIt` in separate invocations. Using a polymorphic interface reference as a formal parameter is a powerful technique — it constrains what the method can accept to objects that implement a specific contract, while remaining open to any concrete type that fulfills it.

Event Processing

KEY CONCEPT

Establishing the relationship between a listener and the component it listens to is accomplished using polymorphism.

In Chapter 6 we examined how GUI events are handled in Java. Recall that responding to an event requires establishing a relationship between an event listener object and a component that can fire the event. This is done by passing the listener to a method that registers it with the component — a process that turns out to be a direct application of polymorphism.

Suppose `MyButtonListener` is a class that acts as an action listener. Setting it up to respond to a `JButton` looks like this:

```
JButton button = new JButton();  
button.addActionListener (new MyButtonListener());
```

Once registered, the listener responds whenever the button fires an action event. Now consider a subtle but important question: the `addActionListener` method was written by engineers at Sun Microsystems years ago, yet we may have written `MyButtonListener` only today. How can a method written in the past accept an object whose class did not yet exist?

The answer is polymorphism. Examining the source of `addActionListener` reveals that its parameter type is `ActionListener` — an interface, not a concrete class. Because the parameter is declared as an interface type, `addActionListener` can accept any object of any class that implements `ActionListener`, regardless of when that class was written. The same principle applies to all other event registration methods.

The `JButton` object itself knows nothing specific about the listener object it receives — only that it implements `ActionListener`. The `JButton` stores the listener and calls the appropriate method on it when the event occurs.

In Chapter 4 we also noted that listeners can be created by extending an adapter class. This is another instance of polymorphism via interfaces. Each adapter class is written to implement the appropriate listener interface, providing empty default implementations for every event-handling method. By extending an adapter, a new listener class automatically inherits the interface implementation from the adapter, satisfying the requirement that it implement the listener interface. The result is the same: polymorphism via interfaces is what enables the listener to be registered and invoked.

However a listener object is created, the underlying mechanism that connects it to its component is polymorphism via interfaces. GUI event handling is one of the most vivid demonstrations of the power and versatility this technique provides.

Polymorphism — whether through inheritance or through interfaces — is a fundamental object-oriented technique that we will continue to apply in appropriate contexts throughout the remainder of this book.

```
// LISTING 6.9
//*****
// Secret.java
//
// Represents a secret message that can be encrypted and decrypted.
//*****

import java.util.Random;

public class Secret implements Encryptable
{
    private String message;
    private boolean encrypted;
    private int shift;
    private Random generator;

    //-----
    // Constructor: Stores the original message and establishes
    // a value for the encryption shift.
    //-----
    public Secret (String msg)
    {
        message = msg;
        encrypted = false;
        generator = new Random();
        shift = generator.nextInt(10) + 5;
    }

    //-----
    // Encrypts this secret using a Caesar cipher. Has no effect if
    // this secret is already encrypted.
    //-----
    public void encrypt ()
    {
        if (!encrypted)
        {
            String masked = "";
            for (int index=0; index < message.length(); index++)
                masked = masked + (char)(message.charAt(index)+shift);
            message = masked;
            encrypted = true;
        }
    }

    //-----
    // Decrypts and returns this secret. Has no effect if this
```



```

// secret is not currently encrypted.
//-----
public String decrypt()
{
    if (encrypted)
    {
        String unmasked = "";
        for (int index=0; index < message.length(); index++)
            unmasked = unmasked +
(char) (message.charAt(index)-shift);
        message = unmasked;
        encrypted = false;
    }
    return message;
}

//-----
// Returns true if this secret is currently encrypted.
//-----
public boolean isEncrypted()
{
    return encrypted;
}

//-----
// Returns this secret (may be encrypted).
//-----
public String toString()
{
    return message;
}
}

```

```

// LISTING 6.10
//*****
// SecretTest.java
//
// Demonstrates the use of a formal interface.
//*****

public class SecretTest
{
    //-----
    // Creates a Secret object and exercises its encryption.
    //-----
    public static void main (String[] args)
    {
        Secret hush = new Secret ("Wil Wheaton is my hero!");
    }
}

```

```
System.out.println (hush);  
hush.encrypt();  
System.out.println (hush);  
hush.decrypt();  
System.out.println (hush);  
}  
}
```

OUTPUT

Wil Wheaton is my hero!

asv*arok~yx*s}*w?*ro|y+

Wil Wheaton is my hero!

Summary of Key Concepts

- A polymorphic reference can refer to different types of objects over time.
 - The binding of a method invocation to its definition is performed at run time for a polymorphic reference.
 - A reference variable can refer to any object created from any class related to it by inheritance.
 - The type of the object, not the type of the reference, determines which version of a method is invoked.
- An interface is a collection of abstract methods and therefore cannot be instantiated.
- Inheritance can be applied to interfaces so that one interface can be derived from another.
- An interface name can be used to declare an object reference variable.
- An interface reference can refer to any object of any class that implements that interface.
- A parameter to a method can be polymorphic, giving the method flexible control of its arguments.
- Establishing the relationship between a listener and the component it listens to is accomplished using polymorphism.